Assignment 1 — Bitemporal Football League Database

Course: Cloud Databases

---

1. Executive summary

A bitemporal relational database modelling the English Premier League 2025–26 season, built in MySQL 8.4 on Docker. The design uses a hand-built approach: anchor tables for stable identity, three version tables carrying both valid time and transaction time, and a `tx_registry` audit log that stamps every logical change with one timestamp.

The headline result is Q6, the three-answer test: the same bitemporal query (`who coached Chelsea on 2026-02-03?`) returns three different coaches depending on *when* the question is asked (Whittaker, Delaney, Whittaker again). This reproduces the pattern from the lecture and proves the implementation handles both temporal axes correctly.

The database holds **137 version rows** across the three bitemporal tables (120 required minimum), **40 logical changes** in `tx_registry`, **65 players**, **9 coaches**, and **6 clubs**.

---

2. Domain and scope

Domain. English Premier League 2025–26. Six clubs (Manchester United, Manchester City, Liverpool, Arsenal, Chelsea, Tottenham Hotspur) with real stadiums. Players and coaches are fictional.

Time window. The database "goes live" on 2025-07-01 09:00 UTC with the pre-season roster. The timeline replays 40 logical changes across 16 months, reaching the start of the 2026–27 pre-season in late October 2026. This spans both the full 2025–26 season (`'2024-25'` and `'2025-26'` are reference entries in the `season` table) and the opening weeks of 2026–27.

What is in scope. Player contracts (relationship + salary), head-coach assignments (role), player loans (relationship variant with gap periods). All three carry full bitemporal history: every change is a close-in-TT + insert-amended + insert-new pattern, never an UPDATE of business data.

What is out of scope. Match results, league standings, player statistics, transfer fees (modelled as salary only). Stadium characteristics are modelled as a non-temporal anchor — real stadiums undergo renovations and capacity changes, but modelling those added complexity without exercising any additional bitemporal mechanics.

---

3. Schema overview

See `er_diagram.png`. The tables fall into three conceptual categories:

Reference / audit (2 tables):

- `tx_registry` — one row per logical change, cited by every version row's `tx_id`
- `season` — named calendar periods used by Q4

Anchors (4 tables, non-temporal):

- `stadium`, `club`, `player`, `coach` — stable identity, no history

Bitemporal version tables (3):

- `contract_version` — player ↔ club relationship, carries `salary_eur`
- `loan_version` — temporary player ↔ (from_club, to_club) relationship
- `head_coach_version` — club ↔ coach role assignment

Each bitemporal table carries four period columns: `valid_from`, `valid_to`, `tx_from`, `tx_to`. All periods are half-open `[from, to)`. Open ends use sentinel values (`9999-12-31` for dates, `9999-12-31 23:59:59.999999` for timestamps) rather than NULL — see Modeling Decisions.

---

4. Modeling decisions

Half-open periods `[from, to)` with sentinels for open ends. `[A, B)` covers A through B−1; adjacent periods `[A, B)` and `[B, C)` tile with no gap or overlap. Gap detection becomes a strict `a.valid_to < b.valid_from`. MySQL has no infinity for dates, so open ends use `'9999-12-31'` for `valid_to` and `'9999-12-31 23:59:59.999999'` for `tx_to` — all temporal predicates become plain comparisons.

DATETIME(6) rather than TIMESTAMP for transaction time. TIMESTAMP caps at 2038 and cannot hold the sentinel; DATETIME(6) extends to 9999 and provides microsecond precision, more than enough for the hours-apart event spacing in the simulated clock.

Anchor + version split. Each domain entity (player, club, coach) exists as a stable anchor row with an immutable surrogate key; version tables carry periods and FK to anchors. "Player 32" never becomes a stale reference, no matter how often their contract or affiliation changes. Stadium and club are also modelled as non-temporal anchors — their attributes don't change meaningfully in our window.

Loans do not close the parent contract. Both `contract_version` (parent) and `loan_version` (loan) carry current rows while a loan is active. This mirrors how federations register loans — the parent retains registration rights. The consequence: I1 (no VT overlap per business key) is enforced per table, not across tables. E9 events (permanent loan conversions — Pierce event 3, Reeves event 10, Berger event 25) must close rows in both tables under the same `tx_id`.

Write discipline: simulated clock and surgical E4 corrections. Transaction timestamps come from a session variable `@sim_now` set before each event, not from `NOW(6)` — the same script run six months apart produces identical bitemporal state. For E4 corrections affecting only one of two currently-believed rows for a business key, we close TT on only that row and insert one replacement (events 22, 32); events 7, 17, 40 affect both rows at a shared boundary and close/replace both.

`tx_registry.reason` **labelled per event.** Every reason starts with the event's E-tag (e.g., `'E6+E3 Jansen gap renewal'`). It's both the audit trail and the input to Q3's `tagged_as` classifier.

---

5. Invariants

I1 — No VT overlap among currently-believed rows per business key. At any moment in transaction time, two currently-believed rows for the same business key must not have overlapping valid periods. Business keys

per table:

- `contract_version` → `player_id` (a player has at most one active contract at any valid instant)
- `loan_version` → `player_id` (no two overlapping loans for the same player)
- `head_coach_version` → `club_id` (each club has at most one head coach at any valid instant — this is also I3)

Enforcement: not a MySQL constraint. Instead, `v_integrity_violations` provides an on-demand check that returns zero rows when the invariant holds. The TT-close-before-INSERT ordering we use within each event guarantees that intermediate state never has two TT-current rows with the same business key, so overlaps cannot form during a transaction.

I2 — Temporal FK (anchor must exist for the version row's valid period). The MySQL FK constraint ensures the anchor row exists at any point in time, but doesn't enforce temporal containment. In our design, anchors are only ever inserted, never deleted, so I2 is upheld in practice.

I3 — Single role holder per central object at any valid instant. For `head_coach_version`, exactly one coach per club at any valid instant. This is I1 applied with `club_id` as the business key — the same `v_integrity_violations` check covers it.

---

6. The timeline in brief

Full narrative in `story-log.md`. Summary:

40 events across 16 months, each with its own `tx_registry` entry and its own `BEGIN TRANSACTION ... COMMIT` block in `03_timeline.sql`. Event tagging coverage:

Tag	Signature	Required	Delivered
E1	Initial load, one tx_id	1	1
E2	Proactive (`valid_from > tx_from`)	3	5
E3	Late entry (`tx_from − valid_from ≥ 7 days`)	5	10
E4	Correction (TT close + replace)	3	5
E5	Real-world end (finite `valid_to`)	5	10
E6	Restart after gap	3	5 (one half-open fencepost — see Failed attempts)
E7	Role handovers across ≥2 central objects, ≥1 corrected	3	3 handovers at 3 clubs
E8	Erroneous insert retracted	1	1
E9	Change touching ≥2 tables, same tx_id	2	3

The headline event is the Chelsea coach handover — Whittaker to Delaney, late-entered on 2026-02-15 and corrected on 2026-02-20 — which produces the three-answer pattern shown in §7 Q6.

---

7. Query results

All queries are implemented as views in `04_views.sql` and exercised by `05_tests.sql`. The `query_params` one-row table holds `valid_at`, `known_at`, and `season_name` for parameterised views (MySQL views cannot take parameters directly). Headline results with `query_params.valid_at = '2026-10-26'`:

Query	Result
Q0 — integrity violations	**0 rows** (all three I1 checks clean)
Q1 — current contracts / coaches / loans	**57 / 6 / 0** = 63 current relationship facts, 3 above the 60 minimum
Q2 — coach tenures (coalesced)	**9** across 6 clubs: 3 with handovers show two tenures each, 3 with stable coaches show one
Q4 — contracts overlapping 2025–26 season	**68 rows** (includes partial-season overlaps)
Q7 — monthly tx_registry histogram	**16 months** (2025-07 → 2026-10), 40 entries total; peaks in Sep 2025, Feb 2026, Sep 2026 (4 each) and Oct 2026 (5, with the buffer-event cluster)

Q1's current head coaches: Harwood/MUN, Reyes/MCI, Conti/LIV, Wagner/ARS, Delaney/CHE, Richter/TOT — one per club.

Q3 — Changes classified by signature

`v_changes` tags every version-row INSERT (excluding E1) with `tagged_as` (from `tx_registry.reason`) and `signature` (from the VT-vs-TT pattern):

- E2 proactive signature: 7 rows (5 true E2 events + 2 incidental — Wagner 1-day proactive, Fletcher 21-day gap renewal).
- E3 late entry signature: 49 rows (over-inclusive — see Failed attempts "signature vs intent").
- E4 corrections (via `tagged_as LIKE 'E4%'`): 5 distinct `tx_id`s — events 7, 17, 22, 32, 40.

Q5 — State at any known_at

Time-travel probes demonstrate full bitemporal recall:

- Arsenal coach as of 2025-10-01: **Francesco Bellini** (before sacking at event 12)
- Arsenal coach as of 2026-01-15: **Erik Wagner** (installed by event 12 on 2025-11-11)
- Chelsea coach as of 2026-02-18 (between the late entry and the correction): **Marcus Delaney** (DB believed handover was Feb 1 at this moment)

Q6 — The three-answer test

The crown jewel. Three identical queries with only `known_at` changing:

```
UPDATE query_params SET valid_at = '2026-02-03', known_at = '2026-02-10
00:00:00.000000';
```

```
SELECT coach_name FROM v_coach_at_club WHERE club_id = 5;  -- → James Whittaker

UPDATE query_params SET known_at = '2026-02-17 00:00:00.000000';
SELECT coach_name FROM v_coach_at_club WHERE club_id = 5;  -- → Marcus Delaney

UPDATE query_params SET known_at = '2026-10-01 00:00:00.000000';
SELECT coach_name FROM v_coach_at_club WHERE club_id = 5;  -- → James Whittaker
```

All three return the expected coaches when run against the current database.

Q8 — Gap detection

Four contract gaps and one loan gap:

Player	Club	Period 1 end	Period 2 start	Gap (days)
Ashworth	MUN	2026-06-30	2026-08-01	32
Fletcher	LIV	2026-07-31	2026-08-15	15
Jansen	LIV	2026-06-30	2026-09-10	72
Kristensen	TOT	2026-06-30	2026-07-01	1

Loan gap: Thornton MCI→ARS, period 1 ends 2025-12-20, period 2 starts 2026-02-25, 67-day gap.

The Kristensen 1-day gap is a half-open fencepost artifact (see Failed attempts). The filtered view `WHERE gap_days >= 7` returns three "meaningful" gaps.

---

8. Performance — the as-of index on contract_version

Part A requires EXPLAIN output of a Q6-style as-of query with and without the composite index `ix_contract_asof (player_id, tx_to, valid_from)` on `contract_version`. The test query is a bitemporal as-of lookup — Pennington's contract state valid on 2026-06-01 as known on 2026-07-01:

```
SELECT * FROM contract_version
WHERE player_id = 1
  AND tx_from   <= '2026-07-01 00:00:00.000000'
  AND '2026-07-01 00:00:00.000000' < tx_to
  AND valid_from <= '2026-06-01'
  AND '2026-06-01' < valid_to;
```

With `ix_contract_asof` — index range scan, three of four predicates pushed into the index via index condition pushdown (`player_id =`, `tx_to >`, `valid_from <=`); only `tx_from` and `valid_to` remain as residual filters. Rows examined: 2.

```
-> Filter: ((tx_from <= '2026-07-01') and ('2026-06-01' < valid_to))  (cost=1.16
rows=0.0741)
```

```
    -> Index range scan on contract_version using ix_contract_asof
       over (player_id = 1 AND '2026-07-01' < tx_to), with index condition
(cost=1.16 rows=2)
```

Without `ix_contract_asof` — `DROP INDEX` fails with error 1553 because the composite's leading column (`player_id`) is also the FK column, so the FK uses the composite as its supporting index. Workaround: add a plain `ix_fk_player (player_id)` first, then drop the composite. In that state the optimiser falls back to an index lookup on the FK index — fetches all rows for player 1 (`rows=3`), then applies all four temporal predicates as post-fetch filters.

```
-> Filter: ((tx_from <= '2026-07-01') and ('2026-07-01' < tx_to)
            and (valid_from <= '2026-06-01') and ('2026-06-01' < valid_to))
(cost=0.755 rows=0.05)
   -> Index lookup on contract_version using ix_fk_player (player_id=1)
(cost=0.755 rows=3)
```

Interpretation. On 65 rows the raw cost estimates (1.16 vs 0.755) actually favour the smaller index — the composite has more pages to traverse on a tiny table. The real finding is qualitative: with the composite, three of four predicates are evaluated inside the scan; without it, all four become residual filters above a lookup that fetches every version for the matched player. The gap grows with versions-per-player.

Side effect of the FK entanglement. Because `ix_contract_asof` serves both the as-of lookup and the FK's index requirement, it does double duty at the storage cost of one index. Any redesign that drops the composite would still need a replacement single-column index on `player_id`, so the "savings" from dropping would be partial.

---

9. Failed attempts

Seven wrong turns caught during review are documented in detail in `story-log.md`. In brief:

- **Three roster collisions** (Pierce double-booked as both E9 permanent transfer and later ARS→CHE loan; Rossi scheduled for E5 release on an already-expired contract; Fletcher planned as E2 instead of E6) were resolved by swapping players or re-tagging events to meet the E5/E6 minimums cleanly.
- **Two tagging errors.** Jansen's gap renewal (event 33) was first tagged `[E3]` only — the 72-day gap between the old E1 contract and the renewal also makes it `[E6]`. Kristensen's "gap-free" renewal (event 26) looked adjacent in calendar terms (2026-06-30 → 2026-07-01) but under half-open semantics leaves 2026-06-30 uncovered, so Q8 correctly flagged it as a 1-day fencepost gap. Both re-tagged to `[E6+E3]`. Lesson: half-open adjacency requires `new.valid_from = old.valid_to`, not `old.valid_to + 1`.
- **A current-facts shortfall** (only 38 facts at wall-clock today, below the 60 minimum) was fixed by shifting 10 silent-expiry contracts to 2028 and adding 15 background-anchor players. Final count: 63.
- **A signature-vs-intent limitation of Q3's classifier.** Data signatures alone cannot distinguish a true E3 late entry from an amended row that preserves an old `valid_from`. 49 rows match the `tx_date - valid_from >= 7 days` signature; only 17 match both the signature AND the `[E3%]` reason tag. The gap is a real property of bitemporal data, not a bug — `tx_registry.reason` is the only reliable discriminator.

10. Scope and limitations

What this design does not do

No distributed scaling. Single-node MySQL. Our workload fits comfortably on one machine (65 players, 137 version rows, kilobytes of data).

No GDPR erasure. Our "never delete" rule means a player's historical rows persist indefinitely. A real system would need a documented retention policy and a tombstone mechanism.

11. Decision guide

Course 2-2 scores temporal implementation approaches across four dimensions. Our hand-built MySQL design rates:

Dimension	Rating	Rationale
VT + TT clocks	**Strong**	Both axes modelled explicitly as four period columns; both queryable
Tamper resistance	**Medium** (with A1)	BEFORE UPDATE/DELETE triggers on all three bitemporal tables forbid mutation of any business column and all DELETEs; stored procedures encapsulate the common write patterns. Ceiling: a DBA with ALTER privileges can still drop the triggers themselves — the unavoidable limit of in-engine protection. See §12
Query ergonomics	**Strong** (with A2)	On MySQL, `query_params` workaround for parameterised views and manual bitemporal predicates (medium). On MariaDB via A2, native `PERIOD FOR` + `WITHOUT OVERLAPS` + `FOR SYSTEM_TIME AS OF` collapse the three-answer query to one declarative line and move engine-enforced I1 to the schema. See §13
Retention control	**Strong**	We never delete. Retention is purely a function of `tx_to` filtering. Views can be defined to hide any history older than a cutoff

The design makes an explicit trade: full bitemporal correctness and auditability in exchange for medium query ergonomics and weak tamper resistance. A1 and A2 are the paths to lift each weakness in turn.

12. A1 — Triggers and stored procedures

File: `sql/06_advanced_a1.sql`. Runs after the main pipeline, does not modify any existing data.

Triggers (6). `BEFORE UPDATE` on each of the three bitemporal tables rejects any statement that (a) changes a business column, (b) modifies a row whose `tx_to` is not the sentinel, or (c) sets `tx_to` outside the open interval `(tx_from, sentinel)`. `BEFORE DELETE` on each table rejects deletes unconditionally. All rejections use `SIGNAL SQLSTATE '45000'` with a descriptive message.

Stored procedures (10). `p_tx_begin(author, reason, sim_now, OUT tx_id)` opens a `tx_registry` entry and returns its id. For each bitemporal table there is `p_<t>_insert(business_cols..., valid_from, valid_to, sim_now, tx_id)`, `p_<t>_close(business_key, sim_now)` which closes all currently-believed rows for that key, and `p_<t>_close_at(version_id, sim_now)` which closes a specific row (for surgical E4 corrections like events 22 and 32).

Verification. The file ends with `CALL p_demo_tamper_update()` and `CALL p_demo_tamper_delete()`, which attempt the forbidden operations inside `DECLARE EXIT HANDLER FOR SQLSTATE '45000'` blocks and report PASS/FAIL. Both PASS on install. A positive demo then runs `p_tx_begin → p_contract_close → p_contract_insert` on Pennington inside a transaction and `ROLLBACK`s, proving the procedure workflow works end-to-end without leaving any persistent change.

Compatibility with the existing pipeline. The triggers permit `UPDATE ... SET tx_to = <value>` on currently-believed rows, which is the only UPDATE shape used in `03_timeline.sql`. The main pipeline continues to run unchanged after A1 is installed — the triggers are additive protection, not a replacement for the existing event scripts.

Effect on §11 decision guide. Tamper resistance moves from weak to medium. A DBA with ALTER privileges can still drop the triggers; that is the ceiling of what in-engine protection can achieve without external audit infrastructure.

---

13. A2 — MariaDB native temporal

File: `sql/06_advanced_a2.sql`. Runs against a separate MariaDB 11.4 service (profile `a2` in `docker-compose.yml`), scoped so the main MySQL pipeline is unaffected. Three demos on a single `head_coach_bitemporal` table that uses `PERIOD FOR validity`, `WITH SYSTEM VERSIONING`, and `UNIQUE (club_id, validity WITHOUT OVERLAPS)`:

Demo 1 — WITHOUT OVERLAPS. Two adjacent half-open periods for the same club are accepted; an overlapping third insert is rejected by the engine with SQLSTATE 23000. Replaces the MySQL discipline of close-before-insert plus the on-demand `v_integrity_violations` check.

Demo 2 — FOR PORTION OF. One statement — `UPDATE ... FOR PORTION OF validity FROM X TO Y SET ... WHERE ...` — replaces the three-statement MySQL pattern (close old row, insert amended row, insert new period). The engine splits the row at the boundary and writes the old half to system-versioning history automatically.

Demo 3 — FOR SYSTEM_TIME AS OF. The MySQL Q6 view carries four bitemporal predicates (`tx_from <= q.known_at`, `q.known_at < tx_to`, `valid_from <= q.valid_at`, `q.valid_at < valid_to`) plus a one-row `query_params` parameter table. On MariaDB the TT axis collapses to one declarative clause and no parameter table is needed:

```
SELECT coach_name FROM head_coach_bitemporal
 FOR SYSTEM_TIME AS OF '2026-02-10'
 WHERE club_id = 5 AND valid_from <= '2026-02-03' AND '2026-02-03' < valid_to;
```

Three probes with `known_at` set to 2026-02-10 / 2026-02-17 / 2026-10-01 return Whittaker / Delaney / Whittaker — the same three-answer pattern the MySQL side produces for Q6.

TT backdating note. MariaDB's system versioning uses the transaction clock for `row_start`, so a straight timeline replay would stamp every row with the actual run date. Demo 3 sets `@@system_versioning_insert_history = 1` for the session, which lets `INSERT` specify `row_start` and `row_end` explicitly. The TIMESTAMP(6) `row_end` max (`'2038-01-19 03:14:07.999999'`) is what MariaDB uses to identify currently-visible rows, so the final belief state is inserted with that explicit `row_end` value. Note that MariaDB requires `TIMESTAMP(6)` (not `DATETIME(6)`) for the system-versioning columns — the MySQL side uses `DATETIME(6)` precisely to escape the 2038 cap, so the two engines take opposite choices on the same decision.

One further MariaDB behaviour surfaced during replay. When `system_versioning_insert_history=1`, `AUTO_INCREMENT` does not advance for history inserts — every row otherwise lands with `version_id=0`. Because the internal primary key for system-versioned tables is effectively `(version_id, row_end)`, two rows sharing the same `row_end` then collide on `(0, row_end)`. Demo 3 avoids this by supplying explicit `version_id` values (101–105) for the five backdated rows. This is a general caution for anyone using history-insert mode to replay a timeline rather than relying on the engine clock.

Effect on §11 decision guide. Query ergonomics moves from medium to strong on the MariaDB side: the three-answer query goes from a bespoke view + parameter table to one declarative line, and I1 moves from a manual check into the schema.

Scope. A2 is a focused demonstration of three native constructs, not a full re-implementation of the 40-event timeline in MariaDB. The constructs shown (`WITHOUT OVERLAPS`, `FOR PORTION OF`, `FOR SYSTEM_TIME AS OF`) are the ones that would collapse the most MySQL code in a full port; the remaining events (E2 proactive, E5 real-world end, E6 restart, E9 cross-table) use the same mechanical close/insert patterns and wouldn't change the ergonomic conclusion.

---

14. Appendix

Project layout, prerequisites, run instructions, and reproduction steps live in the repository `README.md`.